IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 2 November 2025, accepted 25 November 2025, date of publication 2 December 2025, date of current version 16 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3639410

RESEARCH ARTICLE

DSConvNet: A Lightweight Architecture for Extracting Image Features From Depthwise Separable Convolution Network for Edge Devices

SYED MUHAMMAD RAZA[1], SYED MURTAZA HUSSAIN ABIDI[1], MD MASUDUZZAMAN[2], (Member, IEEE), AND SOO YOUNG SHIN[1], (Senior Member, IEEE)

[1]Department of IT Convergence Engineering, Kumoh National Institute of Technology, Gumi 39177, Republic of Korea
[2]Department - Degroote School of Business, McMaster University, Hamilton, ON L8S 4L8, Canada

Corresponding author: Soo Young Shin (wdragon@kumoh.ac.kr)

This work was supported by the IITP (Institute of Information & Coummunications Technology Planning & Evaluation)-ITRC (Information Technology Research Center) grant funded by the Korea government(Ministry of Science and ICT) (IITP-2025-RS-2024-00437190, 50%). This work was supported by the IITP (Institute of Information & Coummunications Technology Planning & Evaluation)-ICAN (ICT Challenge and Advanced Network of HRD) grant funded by the Korea government(Ministry of Science and ICT)(IITP-2025-RS-2022-00156394, 50%).

ABSTRACT This paper presents DSConvNet, a novel architecture based on depthwise separable convolutional blocks for efficient multi-class image classification. Despite progress in compact convolutional neural networks (CNNs), many existing models still impose high computational costs on resource-constrained devices, limiting their real-time applicability. To address this, DSConvNet employs four optimized DSConvNet blocks that reduce parameters, accelerate inference, and stabilize training. Each block combines a 2D convolution, depthwise convolution, batch normalization, and max-pooling, forming an efficient yet expressive feature extractor. The resulting architecture achieves lower complexity, mitigates overfitting, and improves generalization. DSConvNet was evaluated on nine benchmark datasets GTSRB, BTSC, CIFAR-10, CIFAR-100, MNIST, Fashion-MNIST, Imagewoof, Imagenette, and Caltech-101 covering 100 object categories. Without GPU acceleration, the model attained 99.10% accuracy on GTSRB and 98.90% on BTSC, confirming its suitability for real-time edge-based image classification under limited computational resources.

INDEX TERMS Depthwise separable convolution network, image feature extraction, image classification, lightweight architecture, model compression, resource-constrained devices.

I. INTRODUCTION

Image feature extraction is a fundamental problem in computer vision and has been significantly reshaped by the advancements in convolutional neural networks (CNNs). CNNs have transformed visual recognition by self learning hierarchical feature representations directly from raw pixel data. Through the integration of convolutional filters, pooling operations, and non-linear activation functions, CNNs enable robust end-to-end learning while eliminating the dependence on handcrafted features traditionally used in classical vision pipelines [1]. Their strong representational capacity has contributed to state-of-the-art performance across a wide range of applications, including image classification, object detection, image segmentation, traffic-sign recognition, and autonomous driving systems [2], [3]. As these applications increasingly shift toward deployment on resource-limited

The associate editor coordinating the review of this manuscript and approving it for publication was Qiang Li.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

embedded platforms, the design of lightweight yet accurate CNNs has become a crucial research direction.

Despite their success, conventional CNNs exhibit several limitations when deployed in real-time or resource-constrained environments. These limitations stem from the high memory footprint, large parameter counts, and computational demands inherent to standard convolutional layers [4]. Moreover, traditional convolutions operate on local receptive fields, restricting their ability to model global spatial dependencies. The use of fixed kernel sizes (e.g., 3×3) further limits adaptability to varying object scales and image distortions. Increasing network depth or width can partially mitigate these issues, but such expansions introduce substantial computational overhead, diminishing practicality on embedded or mobile hardware [5].

To address these constraints, numerous lightweight CNN architectures have been proposed to balance computational efficiency and representational power. SqueezeNet achieves AlexNet-level accuracy with substantially fewer parameters by employing extensive 1×1 convolutions and aggressive model compression [6]. MobileNetV1 introduced depthwise separable convolutions (DSConv), decomposing standard convolutions into depthwise and pointwise operations to significantly reduce computational complexity [7]. MobileNetV2 augmented this idea with inverted residual blocks and linear bottlenecks to improve information flow and inference efficiency [8], while ShuffleNet applied channel shuffling to enhance information exchange in grouped convolutions [9], [10]. EfficientNet further extended lightweight design principles through compound scaling strategies that jointly optimize network depth, width, and input resolution [11].

Depthwise separable convolutional architectures have emerged as a particularly effective technique for reducing computational cost. By decoupling spatial filtering and cross-channel aggregation first applying a depthwise convolution across individual channels, followed by a pointwise 1×1 convolution these layers drastically reduce both parameter count and floating-point operations (FLOPs) while preserving feature expressiveness [12], [13], [14]. This efficiency makes DSConv-based architectures well suited for multi-class classification and object recognition tasks, especially in edge-computing scenarios [15], [16].

Parallel to these developments, transformer-based architectures such as Vision Transformers (ViTs) have introduced self-attention mechanisms capable of capturing long-range dependencies [17]. Lightweight derivatives, including DeiT and MobileViT, have demonstrated competitive performance on low-power devices [18]. However, these models continue to face challenges in balancing global receptive fields with computational efficiency, maintaining robustness across diverse tasks, and adapting to mobile hardware constraints [19]. Furthermore, although techniques such as quantization, pruning, and knowledge distillation can reduce model complexity [20], they often introduce trade-offs that can degrade robustness and accuracy on edge devices [21], [22]. Recent studies also highlight that many existing lightweight models have not been comprehensively evaluated for feature extraction quality under strict resource limitations [23].

In contrast to prior lightweight architectures such as MobileNetV2, ShuffleNet, and EfficientNet-Lite which commonly depend on complex channel-shuffling strategies, multi-branch structures, or large-scale compound scaling the proposed DSConvNet offers a unified and modular architecture composed of enhanced DSConv blocks equipped with PReLU activations. This design eliminates the need for auxiliary pooling layers or skip connections, thereby reducing implementation overhead and inference latency on edge hardware. A key contribution of this work is demonstrating that a compact, four-block DSConvNet architecture without hardware-specific acceleration can match or outperform widely adopted lightweight models across diverse classification benchmarks, including fine-grained datasets such as BTSC and Caltech-101. Notably, the model achieves over 99% accuracy on GTSRB and over 98% on BTSC using only CPU-based inference.

The contributions of this work are summarized as follows:

- We propose DSConvNet, a lightweight feature-extraction architecture designed specifically for resource-constrained platforms with batch-1 inference efficiency as a primary objective.
- The architecture comprises four depthwise–pointwise DSConv blocks that jointly enable spatial downsampling in the depthwise path and efficient channel mixing in the pointwise path; complete layer-level specifications are provided for reproducibility.
- We conduct extensive evaluations across nine public benchmarks and include on-device experiments to demonstrate the model's deployability under real-world edge constraints.
- We perform targeted ablation studies analyzing the effects of block depth, activation function, kernel/stride choices, input resolution, and pooling strategies, and compare results against strong lightweight baselines under a consistent training and inference pipeline.

The remainder of this article is organized as follows: Section II reviews relevant literature and existing lightweight architectures in image classification. Section III introduces the proposed DSConvNet architecture and details the methodological framework employed. Section IV presents the experimental setup along with a comprehensive analysis of the results to validate the effectiveness of the proposed model. Section V provides an ablation study to examine the impact of architectural design choices on performance. Section VI outlines current limitations, open challenges, and potential directions for future research. Finally, Section VII concludes the study by summarizing key findings and contributions with limitation and future works.

II. LITERATURE REVIEW

Several classical and advanced lightweight models and networks have been proposed [1], [3], [24], [25], [26]. The architecture of conventional deep networks and underscored their limitations in terms of computational complexity and scalability [27]. The first major breakthrough in this direction was Mobilenetv1, which introduced the concept of (DSConv) to decouple spatial and channel-wise correlations, significantly lowering the number of parameters [28]. This innovation enabled the development of lightweight models suitable for mobile and embedded vision applications. However, while DSConvNet reduces computational complexity, it may compromise representational capacity, potentially affecting accuracy in more complex tasks. Building upon this, Mobilenetv2 incorporated inverted residuals and linear bottlenecks, optimizing the flow of information and enhancing inference speed by reducing activation maps at the input and output ends while expanding them in intermediate layers [28]. This architecture improved performance on various tasks, including object detection and semantic segmentation. Nonetheless, the reliance on linear bottlenecks can limit the model's ability to capture non-linear features, which may affect performance in certain scenarios.

ShuffleNetV1 [29] introduced grouped convolutions with channel shuffle to enable inter-group information flow at low computational cost [10]. While effective in reducing MACs, *grouped convolutions* ($G>1$) can degrade single-image latency on specific deployment targets: (i) **mobile/embedded CPUs** where group-wise kernel launches and reduced cache reuse increase memory traffic; (ii) **CPU backends lacking fused/Winograd group-conv kernels** (typical of generic BLAS-based runtimes); and, in some cases, (iii) **mobile NPUs/embedded GPUs** whose schedulers execute groups as separate sub-kernels, limiting parallelism. By contrast, *depthwise* ($G{=}C_{\text{in}}$) and *pointwise* (1×1) convolutions often have specialized kernels on these runtimes, yielding better batch-1 latency. ShuffleNetV2 [30] refined the design with practical guidelines (e.g., minimizing memory access cost and balancing channel widths) to improve deployment efficiency.

GhostNet [31], proposed by Han et al., presented an innovative mechanism for generating multiple Ghost feature maps from a small set of intrinsic features using cheap operations, thereby reducing computational cost without sacrificing accuracy [29]. This design significantly decreased the number of parameters and FLOPs. However, the generation of ghost features through linear operations might limit the diversity of learned representations, potentially impacting performance on complex visual tasks. Similarly, Mobilenetv3, developed via Neural Architecture Search (NAS), leveraged automated design strategies to optimize the network within a vast search space. Although powerful, NAS-based models like EfficientNet and HOTNAS introduce high computational burdens during training due to their reliance on extensive GPU resources [32], [33]. This computational intensity can be a barrier to accessibility and rapid experimentation.

While CNNs focus on local receptive fields, transformer-based models have recently emerged as competitive alternatives for vision tasks by capturing global dependencies through self-attention mechanisms. Initially developed for natural language processing, Vision Transformers (ViTs) adapted these mechanisms to the visual domain and demonstrated strong performance on image classification benchmarks [17]. However, standard ViTs are often resource-intensive, making them unsuitable for real-time or embedded applications. To bridge this gap, several lightweight transformer architectures have been proposed. One such model is MobileViT [34], which combines CNN-style local feature extraction with global attention from transformers, thereby achieving a balance between accuracy and efficiency. Similarly, MobileFormer enables bidirectional information flow between token-based transformer modules and CNN-like feature maps, integrating both local and global representations effectively [35]. Despite these advancements, integrating transformers into resource-constrained environments remains challenging due to the inherent complexity of attention mechanisms. Partial convolution is another novel operator proposed to reduce redundant computation and memory access during training and inference phases, further enhancing transformer efficiency in low-resource settings [36]. While this technique offers promising improvements in efficiency, it may introduce additional complexity in model design and training procedures. These developments highlight an evolving trend toward hybrid architectures that combine the advantages of CNNs and transformers to create compact yet expressive models suitable for edge deployment. Nevertheless, achieving an optimal balance between performance, efficiency, and resource utilization continues to be an active area of research.

A. IMAGE FEATURE EXTRACTION METHODS

Recent advancements in image feature extraction for traffic sign classification have yielded significant improvements in accuracy and efficiency on datasets like GTSRB and BTSC. [37] introduced a lightweight CNN architecture optimized for real-time applications, achieving high accuracy with reduced computational complexity. However, its performance may degrade under varying lighting conditions. Kumar et al. [38] proposed an optimized deep learning model that balances accuracy and processing speed, yet the model adaptability to diverse traffic sign variations remains limited. Yildiz et al. [39] developed a hybrid ensemble method combining multiple CNNs to enhance classification robustness, but the increased model complexity can lead to higher computational demands. Mishra and Goyal [40] focused on effective feature extraction techniques to improve classification performance, though their approach may require extensive preprocessing steps. Sarwatt et al. [41] explored adaptive feature extraction methods tailored for traffic sign recognition, yet the

generalizability of their approach across different datasets needs further validation. Collectively, these studies contribute to the evolving landscape of traffic sign classification, each offering unique solutions while also presenting specific challenges that warrant ongoing research. The comparison of different models on GTSRB and BTSC dataset is shown in Table 2.

III. METHODOLOGY

This section discusses proposed DSConvNet module and architecture of the lightweight image feature extraction module using DSConvNet for a multi-class image classification model is shown in Fig. 2 with a detailed explanation.

A. PROPOSED DSCONVNET BLOCKS

Traditional DSConvNet [42] offers a computationally efficient alternative to standard convolutional operations by factorizing a conventional $k \times k$ convolution into two distinct stages: a depthwise convolution and a pointwise convolution. The depthwise convolution operates independently on each input channel, applying a single spatial filter to extract intra-channel features. The resulting output, shaped as a $c \times n \times n$ tensor where c denotes the number of input channels and $n \times n$ represents the output spatial resolution is then passed to the pointwise convolution stage. This second stage employs 1×1 filters to aggregate channel-wise information, effectively mixing features across different channels. This decomposition substantially reduces the number of parameters and floating-point operations.

Despite the computational efficiency of traditional DSConv, its inherent decoupling of spatial and channel-wise operations restricts early inter-channel feature integration and thus limits representational capacity. In DSConv, each channel is processed independently by the depthwise filter, postponing cross-channel aggregation until the subsequent pointwise convolution. This staged processing, particularly in the initial layers of the network, where the dimensionality of the channel is low, can impede the formation of the rich, discriminative feature hierarchies necessary to capture complex semantic relationships. As a result, model expressiveness, convergence stability, and generalization performance may suffer in complex or fine-grained image classification tasks [43].

Moreover, DSConvNet typically relies on small spatial filters (e.g., 3×3 or 2×2) to minimize computational cost. While efficient, such configurations inherently limit the receptive field of the network [44]. As a result, the model struggles to capture global context or long-range spatial dependencies that are crucial for accurate classification, especially in high intra-class variation datasets like GTSRB or BTSC. Although increasing network depth can partially compensate for this shortcoming, it introduces additional training complexity and the risk of overfitting, especially when applied to small or imbalanced datasets. Another notable limitation of DSConvNet is its lack of built-in regularization and normalization mechanisms.

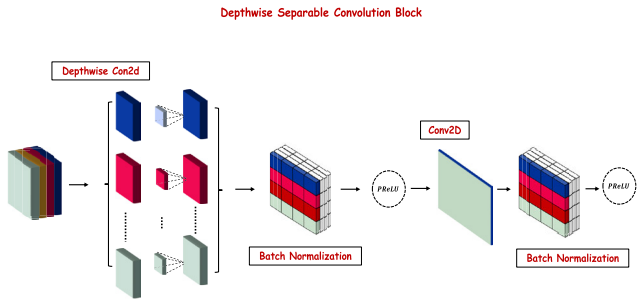

FIGURE 1. Proposed DSConvNet blocks, Depthwise Conv2D ($k \times k$, stride s) → BatchNorm → PReLU→ Pointwise Conv2D (1×1) → BatchNorm → PReLU. Unless otherwise stated, k=2 and s=2 are used for downsampling in the backbone; ablation variants with $k \in \{2, 3\}$ and $s \in \{1, 2\}$ are reported in the experiments. Kernel/stride choices per stage match.

Without consistent use of batch normalization (BN)or spatial pooling, the feature magnitudes across layers can become unstable during training. This instability often increases the model sensitivity to learning rates, initialization schemes, and other hyperparameters, necessitating extensive tuning to achieve reliable convergence. Additionally, the absence of intermediate pooling layers may prevent the network from gaining translation invariance or effectively reducing spatial resolution in a controlled manner. Lastly, many DSConvNet implementations rely solely on ReLU activation functions, which, while computationally simple, suffer from the "dying ReLU" problem due to zero gradients for negative inputs. In compact models where each layer plays a significant role in maintaining information flow, such activation rigidity can hinder learning dynamics. The lack of adaptive non-linearities such as Parametric ReLU (PReLU) further limits the network ability to adjust to complex input distributions, potentially affecting both convergence speed and final accuracy.

These limitations provide the foundation for our proposed enhancements to the traditional DSConvNet structure. In the redesigned architecture, we address key deficiencies in spatial abstraction, channel integration, and training stability. As illustrated in Fig. 1, each proposed DSConvNet block begins with a **depthwise convolution** using a 2×2 kernel and a stride of 2, which applies spatial filtering independently across input channels. This operation is followed by **batch normalization** to stabilize feature distributions and a **PReLU activation** to introduce adaptive non-linearity.

The depthwise convolution operation in the i-th DSConv block is formally defined as:

$$\mathbf{X}_{\text{dw}}^{(i)} = \text{BN}\left(\text{PReLU}\left(\mathbf{W}_{\text{dw}}^{(i)} \odot \mathbf{X}^{(i-1)}\right)\right), \tag{1}$$

where $\mathbf{X}^{(i-1)} \in \mathbb{R}^{H \times W \times C}$ denotes the input tensor to the i-th block, and $\mathbf{W}_{\text{dw}}^{(i)}$ are the depthwise convolution filters. The operator $\odot$ denotes a **channel-wise (depthwise) convolution**, applied independently across the C input channels. BN(·) and PReLU(·) denote batch normalization and Parametric ReLU activation, respectively.

Subsequently, the transformed features $\mathbf{X}_{\text{dw}}^{(i)}$ are processed via a pointwise (1×1) convolution to fuse information

across channels. This operation is again followed by batch normalization and a second PReLU activation:

$$\mathbf{X}_{\text{pw}}^{(i)} = \text{BN}\left(\text{PReLU}\left(\mathbf{W}_{\text{pw}}^{(i)} * \mathbf{X}_{\text{dw}}^{(i)}\right)\right), \tag{2}$$

where $\mathbf{W}_{\text{pw}}^{(i)}$ are the 1×1 filters used for **pointwise convolution**, and $*$ denotes standard 2D convolution over spatial and channel dimensions.

Each DSConv block thus comprises a depthwise convolution for spatial filtering, followed by a pointwise convolution for cross-channel interaction, both regularized through batch normalization and enhanced with adaptive non-linearity via PReLU. The use of stride-2 in the depthwise path enables spatial downsampling, eliminating the need for separate pooling layers and reducing memory overhead.

To perform spatial downsampling and reduce feature map resolution, we apply a 2×2 max pooling operation to the output of the pointwise convolution. The transformation at the i-th DSConv block is defined as:

$$\mathbf{X}^{(i)} = \text{BN}\left(\text{MaxPool}\left(\mathbf{X}_{\text{pw}}^{(i)}, 2\times2\right)\right), \tag{3}$$

where $\mathbf{X}_{\text{pw}}^{(i)}$ is the pointwise-convolved feature map, MaxPool(2×2) denotes a 2×2 spatial max-pooling operation, and BN$(\cdot)$ refers to batch normalization applied post-pooling.

Each DSConv block is modular and repeated four times within the full DSConvNet architecture, progressively reducing spatial resolution while enriching semantic representation. After the convolutional pipeline, the architecture transitions to a fully connected classification head.

This head comprises two dense (fully connected) layers, each containing 512 neurons. Every dense layer is followed by batch normalization, Parametric ReLU (PReLU) activation, and a dropout layer with dropout probability $p = 0.5$ to mitigate overfitting during training.

The PReLU non-linearity used throughout the network is defined as:

$$\text{PReLU}(x) = \begin{cases} x, & \text{if } x > 0, \\ \alpha x, & \text{if } x \leq 0, \end{cases} \tag{4}$$

where $\alpha \in \mathbb{R}$ is a learnable parameter that controls the slope for negative inputs, offering improved flexibility and mitigating the "dying ReLU" effect commonly observed in compact models.

Finally, the output feature map is flattened into a vector $\mathbf{z} \in \mathbb{R}^d$ and passed through a softmax classifier to produce the class probability distribution over C target categories. The classification output is computed as:

$$\hat{\mathbf{y}} = \text{Softmax}\left(\mathbf{W}_{\text{fc}}^{\top}\mathbf{z} + \mathbf{b}_{\text{fc}}\right), \tag{5}$$

where $\mathbf{W}_{\text{fc}} \in \mathbb{R}^{d\times C}$ and $\mathbf{b}_{\text{fc}} \in \mathbb{R}^C$ are the learnable weights and biases of the final dense layer, and $\hat{\mathbf{y}} \in \mathbb{R}^C$ is the resulting class probability vector. The softmax function is defined element-wise as:

$$\text{Softmax}(\hat{y}_j) = \frac{e^{\hat{y}_j}}{\sum_{k=1}^{C} e^{\hat{y}_k}}, \quad j = 1, \ldots, C. \tag{6}$$

TABLE 1. DSConvNet architecture (input 32×32). Each row lists operator, kernel, stride, channels ($C_{\text{in}} \to C_{\text{out}}$), and output tensor size ($H \times W \times C_{\text{out}}$). DSConv($k$, s) = depthwise $k\times k$ (stride s) → BN → PReLU → pointwise 1×1 → BN → PReLU.

Stage	Operator	Kernel	Stride	Channels / Output size
Input	Image	–	–	$32\times32\times3$
Stem	Conv2D + BN	3×3	1	$3\to64$ $32\times32\times64$
Block 1	DSConv(k=2, s=2)	2×2	2	$64\to128$ $16\times16\times128$
Block 2	DSConv(k=2, s=2)	2×2	2	$128\to192$ $8\times8\times192$
Block 3	DSConv(k=2, s=2)	2×2	2	$192\to256$ $4\times4\times256$
Block 4	DSConv(k=2, s=2)	2×2	2	$256\to256$ $2\times2\times256$
Head	Flatten	–	–	1024
	Dense + BN + PReLU	–	–	$1024\to512$
	Dropout(p)	–	–	512
	Dense + Softmax	–	–	$512\to C$

The proposed DSConvNet block design seamlessly integrates lightweight convolutional operations with advanced regularization and adaptive activation mechanisms, resulting in a compact yet expressive architecture. This configuration is particularly well-suited for real-time and edge-deployable image classification systems, offering a balanced trade-off between accuracy and efficiency.

For an input of spatial size $H \times W$ with C_{in} channels, a $k \times k$ *depthwise* conv costs $H \cdot W \cdot C_{in} \cdot k^2$ MACs, while the subsequent *pointwise* 1×1 conv costs $H \cdot W \cdot C_{in} \cdot C_{out}$. Moving from 3×3 to 2×2 reduces the depthwise cost by $\frac{4}{9} \approx 44\%$ at identical resolution. Using stride 2 in the depthwise step replaces separate pooling and fuses downsampling with filtering in a single kernel launch, which reduces memory traffic and improves kernel occupancy on CPUs/NPUs. Although a 3×3 kernel increases receptive field per layer, our ablation shows only marginal accuracy gains compared with its added compute.

1) ACTIVATION RATIONALE

PReLU adds one learnable slope per channel (negligible params) and preserves low-cost, piecewise-linear computation, aligning with our lightweight goal. In contrast, Swish/GELU/Mish introduce non-linearities that increase per-element compute. ReLU6 is quantization-friendly but underfits in our small-capacity regime. Our ablation (Table 6) shows that PReLU achieves the best accuracy-latency trade-off.

B. OVERALL SYSTEM ARCHITECTURE

The proposed lightweight DSConvNet architecture is designed to perform an efficient image feature extraction and image classification task. The architecture consists of four new DSConvNet blocks which includes initial feature extraction, hierarchical learning using new DSConvNet blocks, regularization for generalization with dense classification head. The overall system architecture is shown in Fig. 2. The input image first passes through a 3×3 convolutional layer with 64 filters, followed by batch normalization to stabilize the learning dynamics and extract low-level

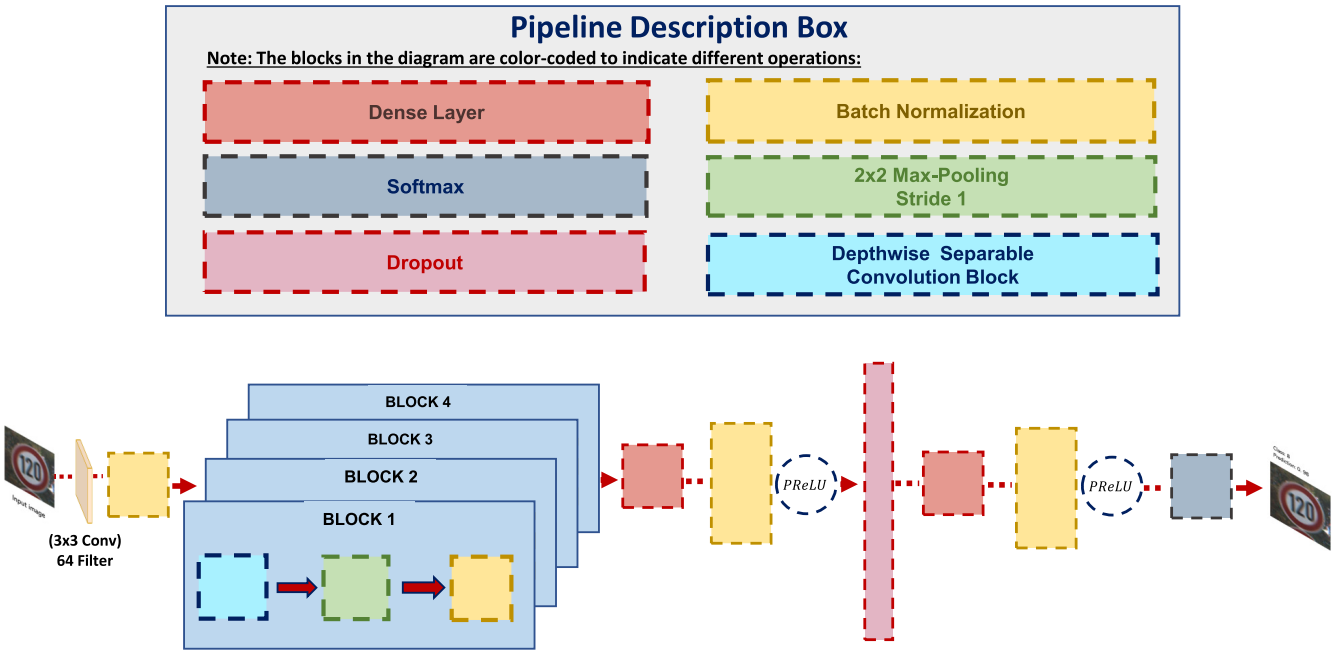

FIGURE 2. DSConvNet Architecture Pipeline. The model begins with a 3×3 convolutional layer ($3\rightarrow64$, stride 1) followed by batch normalization and a 2×2 max-pooling operation to extract low-level features. The core of the network comprises four sequential DSConv blocks (Blocks 1–4). Each block includes a depthwise convolution ($k\times k$, stride 2), batch normalization, and PReLU activation, followed by a pointwise 1×1 convolution, a second batch normalization, and another PReLU. The classifier head consists of a Flatten operation → Dense ($1024\rightarrow512$) → BN → PReLU → Dropout(p) → Dense ($512\rightarrow C$) → Softmax, where C is the number of output classes. The color-coded legend above the pipeline maps each component to its functional role: DSConv blocks (blue), batch normalization (yellow), max-pooling (green), dense layers (red), dropout (pink), softmax (gray), and activation functions (dashed arrows labeled PReLU).

spatial features. These feature maps are then processed by four sequential DSConvNet blocks. Each block comprises a depthwise convolution, pointwise convolution, PReLU activation, batch normalization, and max-pooling, enabling progressively higher-level feature abstraction with minimal parameter overhead. Finally, the aggregated feature maps are flattened and fed into a fully connected classification head with softmax activation, producing a probability distribution over the C target classes from which the highest-probability class is selected.

We adopt PReLU in all convolutional layers. PReLU adds one learnable slope per channel (negligible overhead for tiny models) and mitigates dying-ReLU effects, improving gradient flow without the extra compute of Swish/Mish. Unless stated otherwise, every Conv→BN is followed by PReLU.

IV. EXPERIMENTAL ANALYSIS

This section presents an experimental analysis of the proposed architecture. Experiments were conducted on several benchmark image classification datasets to validate the effectiveness and performance of the proposed model. A complete description of the datasets is provided in the sections.

A. DATASETS

We evaluate DSConvNet on nine publicly available image classification datasets: GTSRB, BTSC, MNIST, Fashion-MNIST, CIFAR-10, CIFAR-100, Imagewoof, Imagenette, and Caltech-101. These datasets collectively span a wide range of domain characteristics, from low-resolution digits and traffic signs to high-resolution fine-grained object classes. All reported results represent the mean performance over **three independent training runs** using different random seeds to ensure statistical reliability.

GTSRB [41] contains 51,839 traffic sign images across 43 classes, with 39,209 training and 12,630 test samples. Image dimensions range from 15×15 to 250×250 pixels; we standardize inputs to 32×32 resolution for compatibility with lightweight models.

BTSC [41] is a more challenging dataset comprising 7,105 samples across 62 classes (4,571 training and 2,534 testing). Though significantly smaller in size than GTSRB, it features a higher number of categories and greater intra-class variation.

MNIST and **Fashion-MNIST** each contain 70,000 grayscale images (28×28 pixels) across 10 classes. MNIST includes handwritten digits, while Fashion-MNIST comprises clothing and accessory categories. Both datasets are commonly used for benchmarking lightweight and efficient deep learning architectures.

CIFAR-10 and **CIFAR-100** each consist of 60,000 color images (32×32, RGB). CIFAR-10 covers 10 broad categories, while CIFAR-100 spans 100 fine-grained classes, making it more challenging for compact models. Both are split into 50,000 training and 10,000 testing samples.

Imagewoof and **Imagenette** are subsets of the larger ImageNet dataset, each containing 10 classes with high-resolution

natural images. Input sizes vary but are standardized to 224×224 pixels. Imagewoof focuses on fine-grained dog breed recognition, while Imagenette includes general object categories (e.g., keyboards, church towers).

Caltech-101 includes approximately 9,000 images distributed across 101 object categories and one background class. Class imbalance and high intra-class variation make it suitable for evaluating model generalization. Images are resized to a uniform resolution of 300×200 pixels for training.

B. DATA AUGMENTATION

Data augmentation is used only on the training set to improve generalization, while validation/test images undergo deterministic resize to $S \times S$ and normalization only. Apply a sequence of stochastic transformations per image and per epoch: RandomResizedCrop to $S \times S$ (scale $\in [0.80, 1.00]$, aspect ratio $\in [0.90, 1.10]$), horizontal flip with probability $p = 0.5$ (vertical flip is disabled, $p = 0$, to preserve semantics), in-plane rotation uniform in $[-15°, +15°]$ (bilinear interpolation, reflect padding), translation up to $\pm 10\%$ of image width/height in x/y, and isotropic zoom with scale $\in [0.90, 1.10]$; an additional mild random crop blueof size $\in [0.90S, S]$ may be applied with $p = 0.3$. Photometric jitter includes brightness shift $\in [-0.20, +0.20]$ and contrast factor $\in [0.80, 1.20]$ (each with $p = 0.8$), plus additive Gaussian noise with $\sigma \in [0.0, 0.02]$ (image scaled to $[0, 1]$) at $p = 0.3$. Finally, all images are normalized with mean $(0.485, 0.456, 0.406)$ and std $(0.229, 0.224, 0.225)$. The operation order is: RandomResizedCrop → flip, rotation, translation, zoom, optional random crop → brightness, contrast, noise → normalize, with parameters sampled independently each epoch. This regimen exposes the model to diverse yet label-preserving variations, thereby reducing overfitting and improving robustness on unseen data.

C. PERFORMANCE METRICS

This section presents a comprehensive evaluation of the proposed DSConvNet architecture using multiple performance metrics, including Top-1 and Top-5 accuracy, sensitivity, F1-score, area under the curve (AUC), and balanced accuracy. These metrics collectively provide a thorough assessment of model classification performance, covering both overall prediction accuracy and its capability to handle class imbalances, detect minority classes, and maintain consistent performance across varying datasets. The combination of these quantitative indicators ensures a reliable and unbiased evaluation of the model in the context of multi-class image classification tasks.

1) TOP-1 AND TOP-5 ACCURACIES

Popular image classification evaluation metrics, such as top-1 and top-5 metrics are used to measure the performance of the trained model. These metrics measure the accuracy of model prediction by considering the top predicted classes for each image. Top-1 accuracy measures the percentage of test samples in which the model highest projected class matches the correct label. It computes the proportion of test samples for which the highest confidence level of the model in its prediction aligns with the actual label. The formula for Top-1 is given in equation shown in 7:

$$\text{Top-1 Accuracy} = \frac{\text{Number of correctly predicted samples}}{\text{Total number of test samples}} \times 100\% \tag{7}$$

For top-5 accuracy is a metric that calculates the percentage of test samples in which the correct label is included in the top-5 predicted classes of the model. The top-5 accuracy metric is more lenient than the top-1 accuracy metric, because it considers whether the correct label is among the top-5 predictions, even if it is not the most confident. The general formula for this is given in eq. 8.

$$\text{Top-5 Accuracy} = \frac{\text{No. of correct top 5 label}}{\text{Total number of test samples}} \times 100\% \tag{8}$$

2) SENSITIVITY

Sensitivity, also referred to as recall, quantifies this capability by measuring the proportion of actual positive samples that the model successfully identifies. This metric provides insight into the model robustness in minimizing false negatives, complementing accuracy-based evaluations.

Mathematically, sensitivity is expressed as:

$$\text{Sensitivity (Recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \times 100\% \tag{9}$$

Based on the experimental results of the proposed DSConvNet architecture specifically, the model achieved a sensitivity of 99.15% on the GTSRB dataset and 98.80% on the BTSC dataset, demonstrating its reliable detection capability in complex, high-variation environments. These results indicate that the proposed DSConvNet not only delivers high overall accuracy but also maintains strong sensitivity, ensuring that critical class instances are accurately detected across varying datasets.

3) F1-SCORE

The F1-score is particularly effective in scenarios with class imbalance or when false positives and false negatives carry significant consequences. It is defined as the harmonic mean of precision and recall, mathematically expressed as:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \times 100 \tag{10}$$

where precision represents the proportion of correctly predicted positive instances among all predicted positives, and sensitivity (recall) reflects the proportion of actual positives that were correctly identified by the model. Based on the experimental results of the proposed DSConvNet architecture, the model demonstrated consistently strong F1-score performance across both evaluation datasets. Specifically, the DSConvNet achieved an F1-score of 99.22% on the GTSRB

dataset and 98.85% on the BTSC dataset. These results underscore the model's ability to maintain an optimal balance between precision and recall, ensuring that the model not only detects positive instances effectively but also minimizes false values.

4) AREA UNDER THE CURVE (AUC)

AUC quantifies the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance. An AUC value of 100% represents perfect classification performance, whereas an AUC of 50% corresponds to random guessing, indicating no discriminative ability.

Mathematically, AUC is computed as the integral of the True Positive Rate (TPR) with respect to the False Positive Rate (FPR), as given in Equation 11:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}) \times 100\% \tag{11}$$

where TPR (True Positive Rate) and FPR (False Positive Rate) are defined as:

$$\text{TPR} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}} \tag{12}$$

$$\text{FPR} = \frac{\text{False Positives (FP)}}{\text{False Positives (FP)} + \text{True Negatives (TN)}} \tag{13}$$

The ROC curves and corresponding AUC scores for the GTSRB and BTSC datasets. The proposed DSConvNet architecture achieved an impressive AUC of 99.70% for the GTSRB dataset and 99.30% for the BTSC dataset. These results demonstrate the model performance to differentiate between classes, reinforcing its robustness and effectiveness in complex, high-variation traffic sign recognition tasks.

5) BALANCE VERSUS ACCURACY

The balance metric provides a class-agnostic evaluation by considering the classification performance across all classes equally. Balanced accuracy is defined as the average of the recall values computed independently for each class. It reflects the model capacity to correctly classify both majority and minority classes without bias. The mathematical formulation of balanced accuracy is given as:

$$\text{Balanced Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i} \times 100 \tag{14}$$

where N denotes the total number of classes, and TP_i and FN_i represent the true positives and false negatives for class i, respectively.

For the proposed DSConvNet architecture, both conventional accuracy and balanced accuracy were evaluated to ensure reliable performance across all classes in the GTSRB and BTSC datasets. On the GTSRB dataset, DSConvNet achieved a Top-1 accuracy of 99.10% and a balanced accuracy of 99.15%. For the BTSC dataset, the model obtained a Top-1 accuracy of 98.90% and a balanced accuracy of 98.70%.

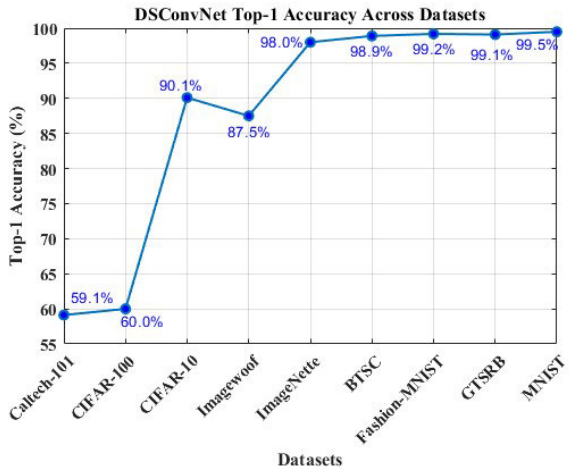

FIGURE 3. Top-1 accuracy of the proposed DSConvNet across nine datasets. From left to right: Caltech-101 (59.12%), CIFAR-100 (60.00%), CIFAR-10 (90.10%), Imagewoof (87.50%), Imagenette (98.00%), BTSC (98.90%), Fashion-MNIST (99.19%), GTSRB (99.10%), and MNIST (99.49%). Each marker corresponds to a single DSConvNet configuration trained per dataset under the protocol in Section IV; numeric labels denote exact percentages.

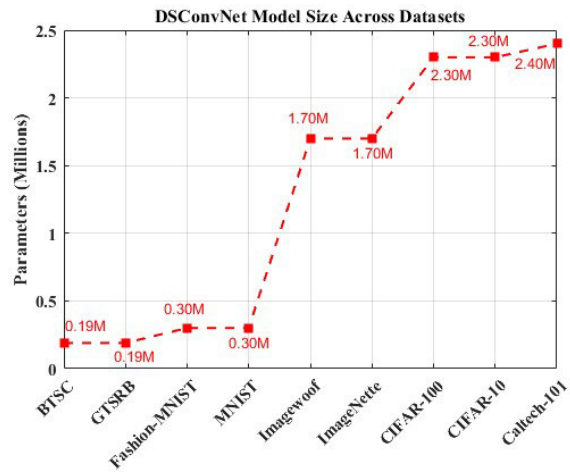

FIGURE 4. Parameter footprint of the proposed DSConvNet across datasets. The curve reports the total number of trainable parameters (in millions) for the configurations used in our experiments: 2.40M (Caltech-101), 2.30M (CIFAR-100), 2.30M (CIFAR-10), 1.70M (Imagewoof), 1.70M (Imagenette), 0.19M (BTSC), 0.30M (Fashion-MNIST), 0.19M (GTSRB), and 0.30M (MNIST).

D. TRAINING

The proposed model framework has been implemented on a CPU, which does not require GPU for training. DSConvNet was trained on the GTSRB and BTSC datasets using standard optimizers such as Adam, AdamW, and SGD optimized with categorical cross-entropy and sparse categorical cross-entropy loss functions. One-hot encoding was applied to the class labels to support multi-class classification. The training process was designed to ensure robust learning and generalization across varying image types and class distributions. The training parameters are,

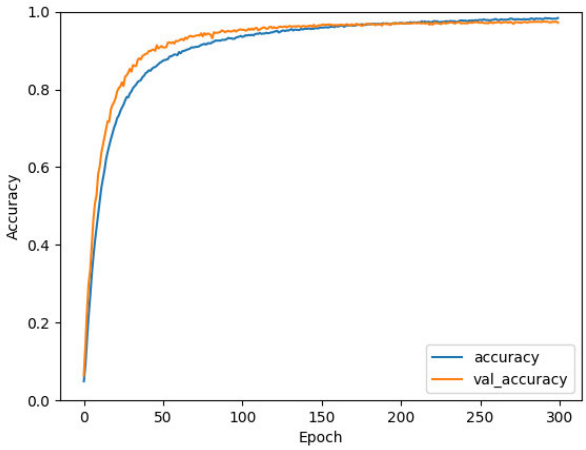

FIGURE 5. Learning curves of DSConvNet on the GTSRB dataset. Top-1 training (blue) and validation (orange) accuracies over 300 epochs show rapid convergence within the first 50 epochs and gradual improvement toward 99%. The close tracking of validation accuracy indicates strong generalization and effective regularization, with no signs of overfitting. The final model is selected based on the highest validation accuracy checkpoint.

batch size 16 32, epochs 300, optimizer AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$), initial LR 1×10^{-3} with cosine annealing (5)-epoch warmup to 1×10^{-3}), label smoothing 0.1. GTSRB/BTSC use input 32×32 (RGB); other datasets follow their standard resolutions. Early stopping patience 20 epochs on validation Top-1. The GTSRB and BTSC datasets have high variations in image dimensions, and to train both datasets, it is essential to find the optimal resolution for the best training. The training was conducted several times to determine the optimal image size as shown in Table 7. The experimental results in Fig. 5 and 6 demonstrate that the model employing 32×32 RGB images attains an accuracy of 99.10% for the GTSRB and 98.90% for the BTSC. To further assess generalization, the proposed model was evaluated across a diverse set of benchmark image classification datasets with varying input resolutions and class complexities. The performance of DSConvNet on the additional benchmark datasets is summarized in Table 2.

E. RESULTS

Despite the simplicity and lightweight structure of the proposed CNN architecture, our technique exhibits stable classification performance. Training on the GTSRB dataset shows an robust classification performance top-1 accuracy of 99.10% and 99.60% of top-5 accuracy, as indicated in Table 1. The BTSC datasets achieved an accuracy of top-1 98.90% and top-5 99.98%; model parameters are 0.245M. The MNIST dataset achieves an accuracy of top-1 99.49% and top-5 99.89%, and Fashion-MNIST achieves an accuracy of top-1 99.19% and top-5 99.79%. Similarly, the numbers in parameters of the MNIST and Fashion-MNIST datasets is 0.3 M. The Imagewoof dataset achieves the accuracy of top-1 87.5% and top-5 91.5% respectively, and the Imagenette dataset achieves the accuracy of top-1 98.0% and top-5 99.0%, respectively. The CIFAR-10 dataset achieves an accuracy of top-1 90.1% and top-5 96.15%, and the CIFAR-100 dataset achieves an accuracy of top-1 60.0% and top-5 85.0%, respectively. Both model parameters are 2.3 M. Finally, the Caltech-101 dataset achieves an accuracy of top-1 59.12% and top-5 80.40%; it contains 2.4 M params. In particular, the accuracy of the model tends to decrease as the number of classes increases. This suggests

TABLE 2. Comparison of lightweight deep learning models on GTSRB and BTSC datasets.

Model	Parameter	Inference Time (ms)	FLOPs(M)
ShuffleNetV1 [29]	3.4	8.07 (GTSRB) 10.9 (BTSC)	140
ShuffleNetV2 [30]	2.3	15.62 (GTSRB) 19.76 (BTSC)	524
GhostNet [31]	5.2	6.67 (GTSRB) 7.9 (BTSC)	167
Mobilenetv1 [28]	4.2	7.02 (GTSRB) 12.85 (BTSC)	149
Mobilenetv2 [30]	3.4	7.96 (GTSRB) 8.9 (BTSC)	300
Mobilenetv3 [32]	2.5	7.25 (GTSRB) 14.24 (BTSC)	356
Deep Thin [45]	0.269	3.43 (GTSRB) 5.6 (BTSC)	60
FreeNet [46]	2.4	13.21 (GTSRB) 25.8 (BTSC)	28
FasterNet [36]	3.9	23.45 (GTSRB) 15.6 (BTSC)	34
STarNet [47]	2.9	33.12 (GTSRB) 25.21 (BTSC)	42
EdgeNeXt [48]	1.3	31.48 (GTSRB) 12.9 (BTSC)	26
DSConvNet(*)	**0.245**	**2.34 (GTSRB)** **4.56 (BTSC)**	**11.37**

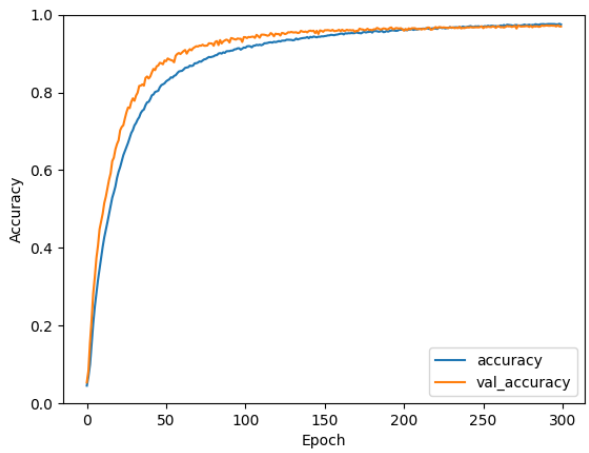

FIGURE 6. Learning curves of DSConvNet on the BTSC dataset. Training (blue) and validation (orange) accuracy curves show consistent convergence, exceeding 85% early and stabilizing near 99% by epoch 220. The validation curve remains tightly coupled with training accuracy throughout, suggesting reliable generalization under significant class imbalance and limited data.

that the proposed model performs exceptionally well for the 100 object categories as shown in Fig. 3 and Fig. 4.

F. COMPARISON WITH SOTA

To validate the efficiency of the proposed framework on edge hardware, experiments were conducted on the **NVIDIA Jetson Orin NX**, a representative platform for real-time embedded AI applications. Table 3 presents the classification accuracy and inference latency of several lightweight convolutional architectures evaluated on two widely used traffic sign recognition benchmarks: GTSRB and BTSC. The results clearly demonstrate that **DSConvNet** achieves superior performance compared to existing state-of-the-art models, including ShuffleNet, GhostNet, and the MobileNet family. On GTSRB, DSConvNet attains an accuracy of **95.7%** with an inference latency of **15.2 ms**, while on BTSC it achieves **87.2%** accuracy with a latency of **18.2 ms**. These results represent a significant improvement over Mobilenetv3, exceeding its BTSC accuracy by more than 10% while offering faster inference than GhostNet and comparable latency to Mobilenetv2.

V. ABLATION STUDY

To rigorously evaluate the robustness and effectiveness of the proposed DSConvNet architecture, an extensive ablation study was conducted, focusing on key architectural components that directly influence model performance. The study systematically investigates the impact of dataset characteristics, input image resolution, kernel size, downsampling strategies (stride versus pooling), and the number of DSConvNet blocks on classification accuracy, inference speed, and generalization performance across benchmark datasets. Additionally, a comprehensive performance comparison against state-of-the-art lightweight CNN architectures is included to contextualize the results.

In particular, varying the number of DSConvNet blocks provides insight into the trade-off between network depth, computational efficiency, and learning stability. Increasing model depth can enhance representational capacity but may introduce additional computational overhead and potential instability during training. Conversely, excessively shallow configurations may reduce model complexity at the expense of reduced feature extraction capability. The following subsections present a detailed, component-wise analysis of these ablation experiments, offering empirical evidence to support the design choices implemented in DSConvNet.

A. NUMBER OF DSCONVNET BLOCKS

The ablation study summarized in Table 4 investigates the influence of varying the number of DSConvNet blocks on the classification accuracy of the proposed DSConvNet across the GTSRB and BTSC benchmark datasets. The results demonstrate a clear trend in which increasing the number of DSConvNet blocks substantially enhances model performance and training stability. On the GTSRB dataset, a configuration with two DSConvNet blocks achieves an accuracy of 76.34%, accompanied by unstable convergence and limited feature representation. Expanding the network to three blocks improves accuracy to 89.15%, though signs of underfitting and convergence instability persist. The optimal performance is attained with four DSConvNet blocks, where DSConvNet achieves 99.10% accuracy, exhibiting stable training dynamics and improved generalization.

A comparable pattern is observed on the BTSC dataset. With only two DSConvNet blocks, the model records a relatively low accuracy of 69.98% and suffers from inconsistent training. Introducing a third block substantially improves accuracy to 92.01%, though minor convergence instability remains. The highest performance is achieved with four DSConvNet blocks, yielding 98.90% accuracy, indicating a well-balanced trade-off between model depth, complexity, and generalization capability. These results emphasize the critical role of architectural depth in enhancing the representational capacity of lightweight CNNs while demonstrating that careful tuning of network depth is essential for achieving both computational efficiency and robust performance.

B. ACTIVATION FUNCTIONS

To assess the robustness of the proposed DSConvNet, Table 6 summarizes Top-1 accuracy on GTSRB/BTSC, parameter count, and batch-1 latency on the edge device for several activation functions. PReLU delivers the best overall trade-off 99.10% (GTSRB) and 98.90% (BTSC) with 0.245 M parameters and 15.2 ms latency. Among parameter-free choices, SiLU (94.9%/92.6%, 15.6 ms) and GELU (94.6%/87.5%, 16.1 ms) improve over ReLU but remain below PReLU. ReLU is the fastest (14.9 ms) but incurs lower accuracy (92.6%/90.3%). Dynamic Tanh yields intermediate accuracy (96.7%/91.4%) and the highest latency (16.8 ms). SeLU performs worst (80.2%/73.9%) despite similar runtime. Based on these results, PReLU is adopted as the default activation in DSConvNet.

C. IMAGE DIMENSION

To examine the effect of input image resolution on the performance of the proposed DSConvNet, an ablation study was conducted using the GTSRB and BTSC datasets with varying image sizes of 240 × 240, 160 × 160, 120 × 120, 60 × 60, and 32 × 32 pixels. The objective was to identify an image resolution that optimally balances model accuracy with computational efficiency. As summarized in Table 7, higher input resolutions did not translate into improved performance. Specifically, DSConvNet exhibited reduced accuracy when trained with 240 × 240 and 160 × 160 pixel images, achieving only 79.40% and 82.69% on GTSRB, and 74.50% and 78.21% on BTSC, respectively. The performance drop at larger resolutions is attributed to the increased computational complexity and model capacity demands, which exceed the lightweight design constraints of DSConvNet, impairing its ability to extract relevant features effectively.

TABLE 3. Accuracy and inference time of lightweight models on Jetson Orin NX. Results are reported on GTSRB and BTSC datasets.

Device	Model	GTSRB Accuracy (%)	BTSC Accuracy (%)	GTSRB Time (ms)	BTSC Time (ms)
Jetson Orin NX	ShuffleNetV1	65.4	51.4	14.1	10.4
	ShuffleNetV2	61.6	53.0	16.4	19.9
	GhostNet	67.4	60.1	21.6	27.9
	Mobilenetv1	70.1	69.6	17.8	20.9
	Mobilenetv2	76.9	65.0	21.8	17.1
	Mobilenetv3	80.2	75.3	25.3	22.3
	DeepThin	86.3	71.8	17.8	19.5
	FreeNet	70.1	71.9	13.5	25.8
	FasterNet	76.3	74.8	23.46	15.6
	StarNet	76.4	74.5	25.21	28.5
	EdgeNeXt	86.7	81.8	20.8	11.5
	DSConvNet	**95.7**	**87.2**	**15.2**	**18.2**

TABLE 4. Ablation study: Effect of number of DSConvNet blocks on accuracy.

Dataset	# Blocks	Accuracy (%)	Observation
GTSRB	2	76.34	Training unstable; poor feature learning
	3	89.15	Improved, but underfitting observed
	4	**99.10**	Optimal; high accuracy and stable loss
BTSC	2	69.98	Inconsistent training; low accuracy
	3	92.01	Better, but convergence still unstable
	4	**98.90**	Best performance; balanced generalization

TABLE 5. Kernel size/downsampling vs. complexity per block (example C_{in}=C_{out}=64).

Design	Params	MACs	Notes
DW 2×2, s=2 + PW	$256 + 4096$	$\propto \frac{HW}{4}(256+4096)$	fused downsampling
DW 3×3, s=2 + PW	$576 + 4096$	$\propto \frac{HW}{4}(576+4096)$	larger DW term
DW 3×3, s=1 + Pool + PW	$576 + 4096$	$\propto HW(576+4096)$ + pool	extra memory op

TABLE 6. Activation ablation on DSConvNet under an identical training/evaluation recipe. Metrics: Top-1 accuracy (%), parameter count (M), and Latency.

Activation	Params (M)	GTSRB Acc. (%)	BTSC Acc. (%)	Latency (ms)
PReLU (per-channel)	**0.245**	**99.10**	**98.90**	**15.2**
ReLU	0.452	92.6	90.3	14.9
SiLU	0.363	94.9	92.6	15.6
GELU	0.636	94.6	87.5	16.1
SeLU	0.998	80.2	73.9	15.0
Dynamic Tanh	0.783	96.7	91.4	16.8

Conversely, reducing the input dimensions to 120 × 120 and 60 × 60 pixels led to a significant improvement in classification accuracy. The model achieved 93.41% and 96.79% on GTSRB, and 87.73% and 94.36% on BTSC, respectively, demonstrating DSConvNet efficiency in processing moderately sized inputs while maintaining low computational overhead. The highest performance was obtained with 32 × 32 pixel inputs, where DSConvNet achieved 99.10% accuracy on the GTSRB dataset and 98.90% on BTSC. These findings indicate that DSConvNet is optimally suited for low-resolution image processing, offering high predictive performance with minimal computational demands, thus reinforcing its applicability for real-time and edge-device scenarios.

TABLE 7. Ablation study: Accuracy of GTSRB and BTSC datasets at various image dimensions.

Dataset	Image Dimension	Accuracy
GTSRB	240 × 240	79.40%
BTSC	240 × 240	74.50%
GTSRB	160 × 160	82.69%
BTSC	160 × 160	78.21%
GTSRB	120 × 120	93.41%
BTSC	120 × 120	87.73%
GTSRB	60 × 60	96.79%
BTSC	60 × 60	94.36%
GTSRB	**32 × 32**	**99.10%**
BTSC	**32 × 32**	**98.90%**

D. KERNEL SIZE

The impact of kernel size on the proposed DSConvNet architecture was systematically evaluated by varying the kernel dimensions within both the standard convolutional layer and the DSConvNet blocks. In the final configuration, a 3 × 3 kernel was employed in the initial convolutional layer to effectively capture fine-grained local features while maintaining a low parameter count. Within each of the four DSConvNet blocks, 2 × 2 depthwise convolution kernels with a stride of 2 were adopted to progressively reduce spatial dimensions, contributing to both parameter reduction and computational efficiency. Extensive empirical

analysis confirmed that this kernel configuration achieves an optimal balance between spatial information extraction and model complexity, making it well-suited for lightweight applications as shown in Table 5.

Alternative configurations with larger kernel sizes, such as 3×3 or 5×5 within the DSConvNet blocks, yielded only marginal improvements in feature representation but introduced significant increases in computational cost and FLOPs, undermining the model suitability for deployment in resource-constrained environments. Conversely, reducing the kernel size below 2×2 severely compromised the network capacity to extract meaningful spatial features, resulting in a notable decline in classification accuracy. Additionally, the inclusion of 1×1 pointwise convolutions within each DSConvNet block was retained, as it facilitates efficient channel-wise information aggregation without introducing substantial computational overhead. The combined effect of 2×2 depthwise and 1×1 pointwise convolutions provides a compact yet expressive feature extraction mechanism, contributing to the overall efficiency and strong performance of DSConvNet.

E. STRIDE VERSUS POOLING

Following the kernel size analysis that established an optimal trade-off between spatial information extraction and model complexity, equal emphasis was placed on evaluating the spatial downsampling strategy within the proposed DSConvNet architecture. Downsampling mechanisms, specifically stride-based operations and pooling layers, directly affect feature resolution, computational efficiency, and overall model stability. In the final design, a stride of 2 was consistently applied within all depthwise convolution layers to reduce spatial dimensions efficiently while maintaining a low parameter count. Complementing this, lightweight max-pooling layers with a 1×1 kernel and stride 1 were integrated after each DSConvNet block. This configuration resulted from extensive ablation experiments comparing alternative downsampling schemes, including aggressive stride-only methods, pooling-exclusive strategies, and various hybrid approaches.

Experimental results revealed that exclusive reliance on stride-based downsampling, although computationally efficient, led to excessive reduction in feature map dimensions, impairing the model's ability to retain fine-grained spatial details crucial for complex visual recognition tasks. This limitation was particularly evident on datasets such as GTSRB and BTSC, where aggressive downsampling resulted in notable performance degradation. Conversely, downsampling through pooling layers alone preserved spatial resolution but introduced increased model size and computational burden, conflicting with DSConvNet lightweight design goals. Moreover, excessive pooling without complementary stride operations led to unstable gradients and slower convergence. The most effective strategy emerged as a hybrid approach, combining moderate stride-based downsampling within DSConvNet blocks with lightweight.

F. PERFORMANCE COMPARISON

To evaluate the efficiency and representational capacity of DSConvNet, we conducted a comprehensive ablation and head-to-head comparison against established SOTA lightweight CNNs models. This comparative evaluation, summarized in Table 2, systematically examines each model predictive accuracy on the benchmark dataset alongside its computational complexity, measured in millions of floating-point operations (MFLOPs). DSConvNet demonstrates a superior balance between computational cost and predictive performance, achieving a Top-1 accuracy of 99.10% with only 11.37M FLOPs. Notably, this outperforms DeepThin [45], which records 98.85% accuracy at 60M FLOPs, as well as Mobilenetv3 and ShuffleNetV2, which require substantially higher computational resources yet deliver lower accuracy.

The performance gap becomes more pronounced when considering earlier MobileNet versions and ShuffleNet variants, which, despite their popularity, exhibit diminishing returns in efficiency, with FLOP counts reaching up to 524M while failing to achieve comparable accuracy levels. GhostNet similarly falls short in balancing model complexity with representational power. These findings underscore the efficacy of DSConvNet architectural enhancements, including the integration of optimized DSConvNet blocks, parametric rectified linear unit (PReLU) activations, and refined normalization schemes. Collectively, these design choices enable DSConvNet to achieve state-of-the-art performance under strict resource-constraints, highlighting its potential for real-time inference and deployment on edge devices.

VI. OPEN CHALLENGES

This section presents the challenges and future opportunities related to the proposed image feature extraction module in different applications and research areas.

A. PERFORMANCE ON OBJECT DETECTION AND IMAGE SEGMENTATION MODELS

An important open challenge arising from this work is the extension of the proposed DSConvNet architecture originally designed for high-performance image classification to more complex computer vision tasks such as object detection and image segmentation. Although the current model demonstrates strong discriminative capability across multi-class classification benchmarks, its applicability to dense prediction tasks remains largely unexplored. To assess feasibility, we outline potential adaptation pathways for both detection and segmentation frameworks.

For **object detection**, DSConvNet can serve as a lightweight backbone generating feature maps at strides 8, 16, 32, which are subsequently consumed by a depthwise-separable FPN/PAN neck to form feature pyramids $P3$, $P4$, $P5$. An anchor-free, YOLO-style detection head can be attached at each pyramid level, comprising classification

and bounding-box regression branches. Training may leverage CIoU or DFL losses for localization and BCE or Focal Loss for classification. To better capture fine scale structures, dilated depthwise convolutions may be inserted at deeper stages to enlarge the effective receptive field, particularly for small-object detection.

For **image segmentation**, two decoder designs are considered. The first is a U-Net–style decoder using skip connections from early and mid-level DSConvNet stages, combined with lightweight depthwise upsampling blocks. The second option is a DeepLabV3+–style decoder with a depthwise Atrous Spatial Pyramid Pooling (ASPP) module employing dilation rates $1, 6, 12, 18$ and a compact edge-refinement head. In both cases, an output stride of 16 (optionally reduced to 8) is adopted. Suitable loss functions include Dice+Cross-Entropy or Focal Loss, depending on class imbalance severity. Pretraining on ImageNet, followed by fine-tuning on datasets such as COCO or Cityscapes with multi-scale training and standard augmentation, further enhances model robustness. During inference, single-scale evaluation can be used, with optional test-time augmentation if latency permits.

B. RESOURCE-CONSTRAINED DEVICES

The target deployment scenarios primarily involve embedded CPUs and mobile SoCs without access to discrete GPUs. Accordingly, DSConvNet incorporates several architectural choices such as compact depthwise–pointwise blocks, narrow bottlenecks, and stride-aware downsampling to reduce activation memory and the number of multiply accumulate operations. Both backbone and decoder widths can be scaled to meet specific device budgets, while the operator set remains compatible with widely used inference frameworks such as NCNN, TFLite, and TensorRT. Mixed-precision execution (FP16) is supported where hardware capabilities allow, and additional quantization can be applied as an optional optimization step to further reduce latency and memory use without significant degradation in accuracy.

C. TRAFFIC SIGN RECOGNITION

Although the proposed DSConvNet achieves high performance on the GTSRB and BTSC benchmarks, traffic sign recognition continues to present distinctive challenges due to variations in illumination, occlusions, environmental noise, and large viewpoint changes. Future work will integrate a lightweight detection front-end for sign localization, followed by DSConvNet-based classification to form a unified detection–classification pipeline. Moreover, task-specific augmentations such as simulated motion blur, glare, weather-induced distortions, and nighttime artifacts will be incorporated to enhance robustness in real-world conditions. Real-time constraints will be addressed through backbone width scaling, judicious input resizing, and latency-aware pruning strategies to ensure deployability on embedded platforms.

VII. CONCLUSION

This paper introduces DSConvNet, a lightweight architecture constructed using depthwise separable convolutional (DSConvNet) blocks, tailored for efficient multiclass image classification across diverse datasets. The network comprises four modular DSConv blocks that effectively extract salient features while substantially reducing parameter count, enabling improved generalization and stability in resource-constrained environments. To enhance non-linearity without introducing computational overhead, the architecture consistently employs Parametric ReLU (PReLU) activations throughout all convolutional stages, including both depthwise and pointwise paths. Max-pooling layers, followed by batch normalization, are incorporated after each DSConv block to improve training stability and mitigate overfitting. DSConvNet was evaluated across a suite of benchmark datasets, including GTSRB and BTSC (without GPU acceleration), and further validated on CIFAR-10, CIFAR-100, MNIST, Fashion-MNIST, Imagewoof, Imagenette, and Caltech-101. The model demonstrates strong classification performance across up to 100 categories while maintaining computational efficiency. These results underscore DSConvNet potential as a deployable solution for real-time, edge-based vision tasks. Future work may explore scaling the architecture to larger datasets such as Caltech-256 and ImageNet, as well as extending its utility to dense prediction tasks including object detection and segmentation.

LIMITATIONS AND FUTURE WORK

While DSConvNet achieves competitive accuracy with significantly fewer parameters, its design favors classification tasks with moderate image resolution and balanced class distributions. The current architecture, although effective across benchmark datasets, has not yet been validated on large-scale datasets such as ImageNet or on tasks beyond classification, such as object detection or semantic segmentation. Additionally, the exclusive use of PReLU in all nonlinear layers, while beneficial for convergence, may not yield similar advantages across all hardware platforms or datasets. Future work will explore the integration of lightweight attention mechanisms and adaptability to diverse deployment scenarios including mobile and embedded vision systems.

REFERENCES

[1] A. Dhillon and G. K. Verma, "Convolutional neural network: A review of models, methodologies and applications to object detection," *Prog. Artif. Intell.*, vol. 9, no. 2, pp. 85–112, Jun. 2020.

[2] Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: Analysis, applications, and prospects," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 6999–7019, Dec. 2022.

[3] S. M. Raza, S. M. H. Abidi, M. Masuduzzaman, and S. Y. Shin, "Lightweight deep learning for visual perception: A survey of models, compression strategies, and edge deployment challenges," *Neurocomputing*, vol. 656, Dec. 2025, Art. no. 131357.

[4] A. Younesi, M. Ansari, M. Fazli, A. Ejlali, M. Shafique, and J. Henkel, "A comprehensive survey of convolutions in deep learning: Applications, challenges, and future trends," *IEEE Access*, vol. 12, pp. 41180–41218, 2024.

[5] Y. Zhou, S. Chen, Y. Wang, and W. Huan, "Review of research on lightweight convolutional neural networks," in *Proc. IEEE 5th Inf. Technol. Mechatronics Eng. Conf. (ITOEC)*, Jun. 2020, pp. 1713–1720.

[6] R. Bhuvaneswari and K. K. Enaganti, "Robust image forgery classification using SqueezeNet network," in *Proc. 1st Int. Conf. Adv. Electr., Electron. Comput. Intell. (ICAEECI)*, Oct. 2023, pp. 1–5.

[7] E. Elfatimi, R. Eryigit, and L. Elfatimi, "Beans leaf diseases classification using MobileNet models," *IEEE Access*, vol. 10, pp. 9471–9482, 2022.

[8] K. Dong, C. Zhou, Y. Ruan, and Y. Li, "MobileNetV2 model for image classification," in *Proc. 2nd Int. Conf. Inf. Technol. Comput. Appl. (ITCA)*, Dec. 2020, pp. 476–480.

[9] A. Krueangsai and S. Supratid, "Effects of shortcut-level amount in lightweight ResNet of ResNet on object recognition with distinct number of categories," in *Proc. Int. Electr. Eng. Congr. (iEECON)*, Mar. 2022, pp. 1–4.

[10] D.-Q. Vu, N. T. Le, and J.-C. Wang, "(2+ 1) d distilled shufflenet: A lightweight unsupervised distillation network for human action recognition," in *Proc. 26th Int. Conf. Pattern Recognit. (ICPR)*, Aug. 2022, pp. 3197–3203.

[11] S. Ghosh, M. J. Mondal, S. Sen, S. Chatterjee, N. Roy, and S. Patnaik, "A novel approach to detect and classify fruits using ShuffleNet V2," in *Proc. IEEE Appl. signal Process. Conf. (ASPCON)*, Oct. 2020, pp. 163–167.

[12] J.-H. Luo and J. Wu, "Neural network pruning with residual-connections and limited-data," in *Proc. IEEE/CVF Conf. Comput. Vis. pattern Recognit.*, Jun. 2019, pp. 1458–1467.

[13] G. Lu, W. Zhang, and Z. Wang, "Optimizing depthwise separable convolution operations on GPUs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 1, pp. 70–87, Jan. 2022.

[14] J. Choi, H. Sim, S. Oh, S. Lee, and J. Lee, "MLogNet: A logarithmic quantization-based accelerator for depthwise separable convolution," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 12, pp. 5220–5231, Dec. 2022.

[15] B. Li, H. Wang, X. Zhang, J. Ren, L. Liu, H. Sun, and N. Zheng, "Dynamic dataflow scheduling and computation mapping techniques for efficient depthwise separable convolution acceleration," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 8, pp. 3279–3292, Aug. 2021.

[16] F. Kong, K. Hu, Y. Li, D. Li, X. Liu, and T. S. Durrani, "A spectral–spatial feature extraction method with polydirectional CNN for multispectral image compression," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 2745–2758, 2022.

[17] C.-F.-R. Chen, Q. Fan, and R. Panda, "CrossViT: Cross-attention multi-scale vision transformer for image classification," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 347–356.

[18] A. Wang, H. Chen, Z. Lin, H. Pu, and G. Ding, "RepViT: Revisiting mobile CNN from ViT perspective," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2023, pp. 15909–15920.

[19] H.-I. Liu, M. Galindo, H. Xie, L.-K. Wong, H.-H. Shuai, Y.-H. Li, and W.-H. Cheng, "Lightweight deep learning for resource-constrained environments: A survey," *ACM Comput. Surveys*, vol. 56, no. 10, pp. 1–42, Oct. 2024.

[20] J. Qiu, Y. Zhao, and W. Li, "Neural architecture search method based on improved Monte Carlo tree search," in *Proc. China Autom. Congr. (CAC)*, 2023, pp. 9033–9037.

[21] Y. Liu and J. Lu, "Double loss block neural architecture search," in *Proc. IEEE 10th Joint Int. Inf. Technol. Artif. Intell. Conf. (ITAIC)*, Jun. 2022, pp. 731–734.

[22] C.-H. Wang, K.-Y. Huang, Y. Yao, J.-C. Chen, H.-H. Shuai, and W.-H. Cheng, "Lightweight deep learning: An overview," *IEEE Consum. Electron. Mag.*, vol. 13, no. 4, pp. 51–64, Jul. 2024.

[23] Y. Liu, H. Liang, and S. Zhao, "LMSFF: Lightweight multi-scale feature fusion network for image recognition under resource-constrained environments," *Expert Syst. Appl.*, vol. 262, Mar. 2025, Art. no. 125584.

[24] W. Rawat and Z. Wang, "Deep convolutional neural networks for image classification: A comprehensive review," *Neural Comput.*, vol. 29, no. 9, pp. 2352–2449, Sep. 2017.

[25] Q. Liu, N. Zhang, W. Yang, S. Wang, Z. Cui, X. Chen, and L. Chen, "A review of image recognition with deep convolutional neural network," in *Proc. Int. Conf. Intell. Comput.*, 2017, pp. 69–80.

[26] S. Rehman, H. Ajmal, U. Farooq, Q. U. Ain, F. Riaz, and A. Hassan, "Convolutional neural network based image segmentation: A review," *Proc. SPIE*, vol. 10649, pp. 191–203, Apr. 2018.

[27] N. Aloysius and M. Geetha, "A review on deep convolutional neural networks," in *Proc. Int. Conf. Commun. Signal Process. (ICCSP)*, Apr. 2017, pp. 0588–0592.

[28] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[29] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet v2: Practical guidelines for efficient CNN architecture design," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 122–138.

[30] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.

[31] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in *Proc. IEEE/CVF Conf. Comput. Vis. pattern Recognit.*, Jun. 2019, pp. 1580–1589.

[32] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.

[33] J. Yang, Y. Liu, and H. Xu, "HOTNAS: Hierarchical optimal transport for neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11990–12000.

[34] S. Mehta and M. Rastegari, "MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer," 2021, *arXiv:2110.02178*.

[35] Y. Chen, X. Dai, D. Chen, M. Liu, X. Dong, L. Yuan, and Z. Liu, "Mobile-former: Bridging MobileNet and transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 5260–5269.

[36] J. Chen, S.-h. Kao, H. He, W. Zhuo, S. Wen, C.-H. Lee, and S.-H. G. Chan, "Run, don't walk: Chasing higher flops for faster neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2023, pp. 12021–12031.

[37] J. Zhang, W. Wang, C. Lu, J. Wang, and A. K. Sangaiah, "Lightweight deep network for traffic sign classification," *Ann. Telecommun.*, vol. 75, nos. 7–8, pp. 369–379, Aug. 2020.

[38] M. Kumar, S. Ramalingam, and A. Prasad, "An optimized intelligent traffic sign forecasting framework for smart cities," *Soft Comput.*, vol. 27, no. 23, pp. 17763–17783, Dec. 2023.

[39] G. Yildiz, A. Ulu, B. Dizdaroglu, and D. Yildiz, "Hybrid image improving and CNN (HIICNN) stacking ensemble method for traffic sign recognition," *IEEE Access*, vol. 11, pp. 69536–69552, 2023.

[40] J. Mishra and S. Goyal, "An effective automatic traffic sign classification and recognition deep convolutional networks," *Multimedia Tools Appl.*, vol. 81, no. 13, pp. 18915–18934, May 2022.

[41] D. Sebastian Sarwatt, F. Kulwa, J. Ding, and H. Ning, "Adapting image classification adversarial detection methods for traffic sign classification in autonomous vehicles: A comparative study," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 19046–19061, Nov. 2024.

[42] Z. Y. Khan and Z. Niu, "CNN with depthwise separable convolutions and combined kernels for rating prediction," *Expert Syst. Appl.*, vol. 170, May 2021, Art. no. 114528.

[43] J.-G. Jang, C. Quan, H. D. Lee, and U. Kang, "Falcon: Lightweight and accurate convolution based on depthwise separable convolution," *Knowl. Inf. Syst.*, vol. 65, no. 5, pp. 2225–2249, May 2023.

[44] Q. Qu, X. Chen, V. Chung, and Z. Chen, "Light field image quality assessment with auxiliary learning based on depthwise and anglewise separable convolutions," *IEEE Trans. Broadcast.*, vol. 67, no. 4, pp. 837–850, Dec. 2021.

[45] W. A. Haque, S. Arefin, A. S. M. Shihavuddin, and M. A. Hasan, "DeepThin: A novel lightweight CNN architecture for traffic sign recognition without GPU requirements," *Expert Syst. Appl.*, vol. 168, Apr. 2021, Art. no. 114481.

[46] H. Yu, H. Chen, W. Peng, X. Cheng, and G. Zhao, "FreeNet: Liberating depth-wise separable operations for building faster mobile vision architectures," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 9, pp. 9607–9615.

[47] X. Ma, X. Dai, Y. Bai, Y. Wang, and Y. Fu, "Rewrite the stars," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 5694–5703.

[48] M. Maaz, A. Shaker, H. Cholakkal, S. Khan, S. W. Zamir, R. M. Anwer, and F. S. Khan, "EdgeNeXt: Efficiently amalgamated CNN-transformer architecture for mobile vision applications," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 3–20.

SYED MUHAMMAD RAZA received the M.S. degree in IT convergence engineering from the WENS Laboratory, Kumoh National Institute of Technology (KIT), South Korea, in 2023. He is currently a Graduate Fellow and a Researcher with KIT. He is an AI Software Engineer with 3i Inc. His research interests include image processing, deep learning, lightweight deep learning models, and computer vision applications.

SYED MURTAZA HUSSAIN ABIDI received the M.S. degree in IT convergence engineering from the WENS Laboratory, Kumoh National Institute of Technology (KIT), South Korea, in 2023. He is currently a Graduate Fellow and a Researcher with KIT. His research interests include image processing, deep learning, lightweight deep learning models, and computer vision applications.

MD MASUDUZZAMAN (Member, IEEE) received the B.Sc. and M.Sc. degrees in computer science from American International University-Bangladesh (AIUB), solidifying his academic foundation in computational theory and applied technologies, and the Ph.D. degree in IT convergence engineering from Kumoh National Institute of Technology (KIT), South Korea, demonstrating a deep commitment to interdisciplinary research and innovation. Prior to this role, he held a similar position with KIT, for one year. Earlier in his career, he was a Lecturer with AIUB for four years, contributing to academic instruction and research development. He is currently a Postdoctoral Fellow with McMaster University, Canada. His primary research interests span a broad spectrum of contemporary and emerging fields, including blockchain technology, cryptography, deep learning, edge computing, the Internet of Things (IoT), networks security, and autonomous vehicles.

SOO YOUNG SHIN (Senior Member, IEEE) received the Ph.D. degree in electrical engineering and computer science from Seoul National University, in 2006. He was with the WiMAX Design Laboratory, Samsung Electronics, Suwon, South Korea, from 2007 to 2010. He was a Postdoctoral Researcher with the University of Washington, USA, in 2007. He joined as a full-time Professor with the School of Electronics, Kumoh National Institute of Technology, Gumi, South Korea, in 2010. In addition, he was a Visiting Scholar with The University of British Columbia, Canada, in 2017. He is currently an Associate Professor. His research interests include 5G/6G wireless communications and networks, signal processing, the Internet of Things (IoT), mixed reality, drone applications, and related areas.